

**Name: Shahd Mahmoud**

## **Clock Divider (Even & Odd 50% Duty Cycle) Design & Verification**

### **PART 1: Initial Implementation (Non-50% Duty Cycle for Odd Ratios)**

#### **1. Introduction and Objective**

Clock dividers are fundamental components in digital systems used to derive lower-frequency clocks from a reference clock source. The initial objective was to design a configurable clock divider module (Clk\_div) in Verilog HDL supporting variable integer division ratios (i\_div\_ratio) for both Even and Odd values.

#### **2. Architecture & Design Approach (Single-Edge Toggling)**

The first iteration relied exclusively on positive-edge clocking (posedge i\_ref\_clk) with an internal counter, a state toggle register (o\_reg\_div\_clk), and an alternating control flag (Flag).

- **Even Division Logic:** The threshold is defined by  $\text{Half\_Even} = \frac{\text{Div\_Ratio}}{2}$

The output toggles every (Half\_Even - 1) counts, resulting in a symmetrical square wave.

- **Odd Division Logic:** Since an odd integer cannot be evenly divided by 2 using whole clock cycles, the counter toggles the output using two alternating limits:

$$\text{Half\_Even} = \lfloor \text{Div\_Ratio} / 2 \rfloor$$

$$\text{Half\_Odd} = \text{Div\_Ratio} - \text{Half\_Even}$$

- **State Switching:** A register Flag alternates between 1 and 0 on each toggle, switching the reset condition between `counter == Half_Even - 1` and `counter == Half_Odd - 1`.

### 3. Mathematical Analysis of the Duty Cycle Limitation

While this approach successfully generates the correct target frequency for both Even and Odd division ratios, it fails to achieve a **50% duty cycle for Odd numbers**.

Because all state transitions occur strictly on the rising edge of `i_ref_clk`, the HIGH time and LOW time must both be integer multiples of the reference clock period (`T_clk`):

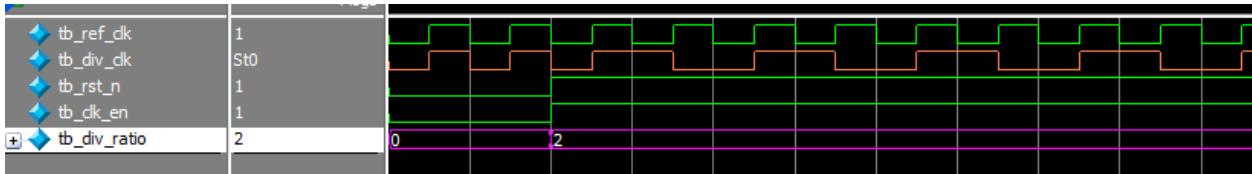
- **Divide-by-3 (Div\_Ratio = 3):**
  - HIGH duration =  $2 * T_{clk}$
  - LOW duration =  $1 * T_{clk}$
  - **Duty Cycle:**  $2/3 * 100\% = 66.7\%$  (or 33.3% depending on initial polarity).
- **Divide-by-5 (Div\_Ratio = 5):**
  - HIGH duration =  $3 * T_{clk}$
  - LOW duration =  $2 * T_{clk}$
  - **Duty Cycle:**  $3/5 * 100\% = 60.0\%$  (or 40.0%).

#### Why This Fails Standard Protocol Requirements

In high-speed SoC design, clock signals driving synchronous logic require a 50% duty cycle to ensure equal setup and hold time margins for both positive-edge and negative-edge triggered registers.

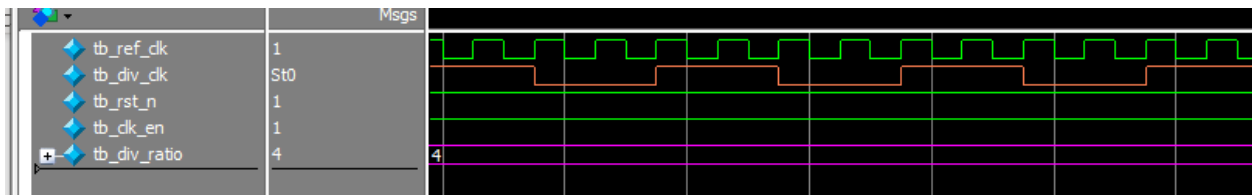
## 4. Initial Design Simulation & Waveform Analysis

- **Figure 1.1: Even Division (Divide-by-2 Waveform)**



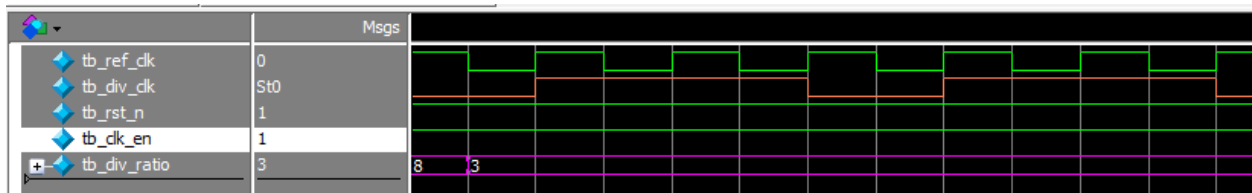
*Observation:* Shows a symmetric output with 1cycles HIGH and 1 cycles LOW (Exact 50% Duty Cycle).

- **Figure 1.2: Even Division (Divide-by-4 Waveform)**



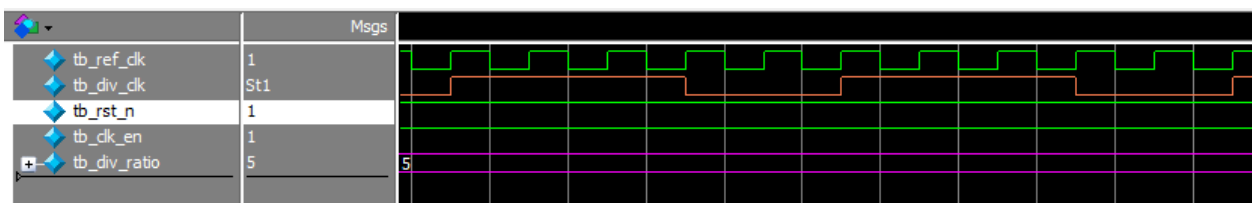
*Observation:* Shows a symmetric output with 2 cycles HIGH and 2 cycles LOW (Exact 50% Duty Cycle).

- **Figure 1.3: Odd Division Initial Attempt (Divide-by-3 Waveform)**



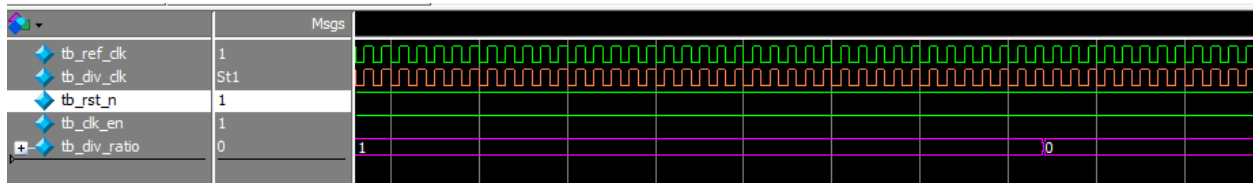
*Observation:* Shows 2 cycles HIGH and 1 cycle LOW, confirming the non-50% duty cycle limitation (66.7% Duty Cycle).

- **Figure 1.4: Odd Division Initial Attempt (Divide-by-5 Waveform)**



*Observation:* Shows 3 cycles HIGH and 2 cycles LOW (60% Duty Cycle)

**Figure 1.5: Bypass Mode / Zero-Division Test (Divide-by-0 or Divide-by-1 Waveform)**



- **Configuration:** i\_div\_ratio = 8'h00 or 8'h01, i\_clk\_en = 1
- **Observation:** The internal signal CLK\_DIV\_EN stays 0 because the module enforces (i\_div\_ratio > 1'b1). As a result, the internal counter remains frozen at 0, and the output multiplexer bypasses the divider logic completely, routing i\_ref\_clk directly to o\_div\_clk.
- **Result:** o\_div\_clk perfectly mirrors i\_ref\_clk with zero clock division delay and maintains the original reference clock's duty cycle.

# PART 2: Enhanced Architecture (True 50% Duty Cycle for Even & Odd Ratios)

## 1. Objective of the Redesign

To redesign the Clk\_div module to guarantee a **strict 50% duty cycle** across **all division ratios** (Even and Odd) while ensuring full RTL synthesis compliance and zero STARCO5 lint violations.

## 2. Deep-Dive: How True 50% Duty Cycle is Achieved for Odd Ratios

To achieve a 50% duty cycle for an odd integer  $N$ , the output clock must remain HIGH for exactly  $N/2$  clock periods ( $1.5 * T_{clk}$ ) for divide-by-3, ( $2.5 * T_{clk}$ ) for divide-by-5). Because positive edges occur only at full integer periods (1.0, 2.0, 3.0.....), generating a fractional  $0.5 * T_{clk}$  phase pulse requires utilizing **both positive and negative clock edges**.

### Step-by-Step Hardware Mechanism:

#### 1. Step 1: Counter Generation

A counter counts continuously from **0** to **i\_div\_ratio - 1** on **posedge i\_ref\_clk**.

#### 2. Step 2: Positive-Edge Pulse (q1)

A comparative condition  $d1 = (\text{counter} < (\text{i\_div\_ratio} >> 1))$  is fed into a positive-edge D-Flip-Flop (q1).

- For **N = 3**: d1 is HIGH for 1 clock cycle (when counter == 0).
- q1 output remains HIGH for **1 full clock cycle** on **posedge i\_ref\_clk**.

#### 3. Step 3: Negative-Edge Phase Shift (q2)

The output q1 is sampled by a second D-Flip-Flop (q2) clocked on the **falling edge** (**negedge i\_ref\_clk**).

- This creates an identical waveform q2 that is phase-shifted (delayed) by exactly **half a clock cycle (  $0.5 * T_{clk}$  )** relative to q1.

#### 4. Step 4: Phase Combination (OR Gate)

For odd division ratios, the final output is produced by combining q1 and q2 using a bitwise OR operation:

$$o\_div\_clk_{odd} = q1 \mid q2$$

### 3. Mathematical Proof of 50% Duty Cycle:

- **Pulse Width Extension:**

$$\text{HIGH Duration} = \text{Width}(q1) + \text{Phase Shift}(q2) = \lfloor N/2 \rfloor + 0.5 = \frac{N}{2} \text{ cycles}$$

- **Divide-by-3 Example (N=3):**

- HIGH Duration =  $1 + 0.5 = 1.5$  cycles
- LOW Duration =  $3 - 1.5 = 1.5$  cycles
- Duty Cycle =  $1.5/3.0 = 50.0\%$

- **Divide-by-5 Example (N=5):**

- HIGH Duration =  $2 + 0.5 = 2.5$  cycles
- LOW Duration =  $5 - 2.5 = 2.5$  cycles
- Duty Cycle =  $2.5/5.0 = 50.0\%$

### 4. Linting Violation Resolution (STARC05-1.3.1.3 Rule)

During SpyGlass static analysis, the initial output assignment generated a STARC05 rule violation: **STARC05-1.3.1.3: Asynchronous reset signal used as non-reset signal.**

#### Cause & Correction

The reset signal i\_rst\_n was directly referenced inside a combinational assign statement:

```
// Violating Code:
assign o_div_clk = (!CLK_DIV_EN || !i_rst_n) ? i_ref_clk : o_reg_div_clk;
```

## Fix Applied:

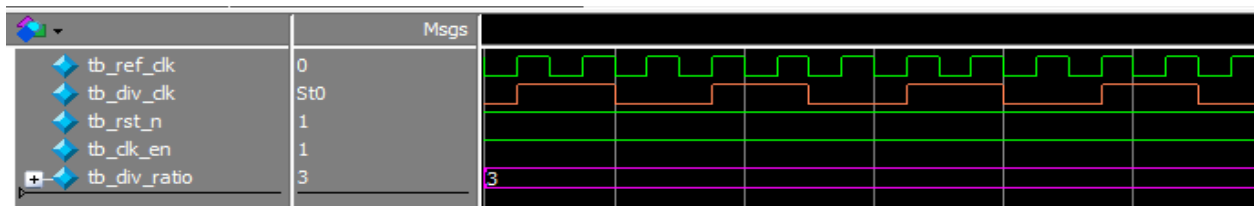
The reset signal `i_rst_n` was removed from the combinational Mux and restricted strictly to sequential flip-flop reset inputs. When `i_rst_n` is asserted low, all internal registers (counter, `q1`, `q2`) clear synchronously/asynchronously within their respective always blocks, safely forcing the output to 0 without placing `i_rst_n` in the data path:

```
// Corrected Code:  
assign o_div_clk = (!CLK_DIV_EN) ? i_ref_clk : (Odd ? (q1 | q2) : q1);
```

## 5. Enhanced Verification Results & Waveforms

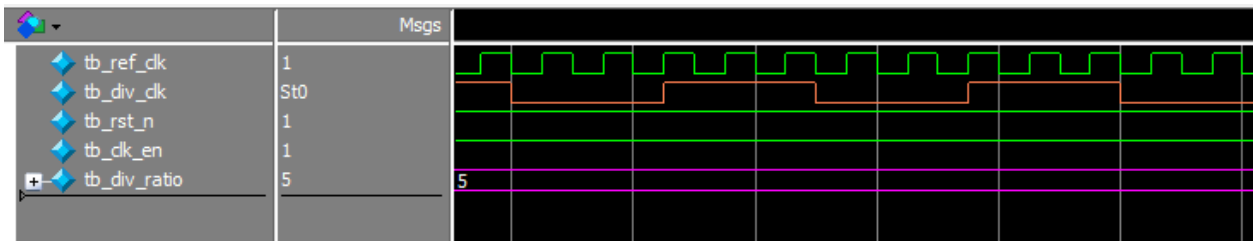
Simulation cases were executed in ModelSim to confirm the redesigned dual-edge architecture:

- **Figure 2.1: Corrected Divide-by-3 Waveform (True 50% Duty Cycle)**



*Analysis:* Demonstrates `q1` (pos-edge), `q2` (neg-edge phase shift), and the resulting `o_div_clk` staying HIGH for exactly 1.5 input cycles and LOW for 1.5 input cycles.

- **Figure 2.2: Corrected Divide-by-5 Waveform (True 50% Duty Cycle)**



*Analysis:* Confirms that for Even ratios, `q1` bypasses the OR gate directly to provide a clean 2-cycle HIGH / 2-cycle LOW output.